KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

COURIER PARCEL TRACKING AND DELIVERY STATUS

Submitted by

KOWSIKA M

25BIT027

Under the Guidance of

DHILEEBAN A

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **I**

DECLARATION

I hereby declare that the project entitled **" COURIER PARCEL TRACKING AND DELIVERY STATUS "** submitted in partial fulfillment of the requirements for the course **[B.Sc.IT I-A]** is a **record of my original work** carried out under the guidance of **[DHILEEBAN A]**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

CERTIFICATE

This is to certify that the project entitled **"COURIER PARCEL TRACKING AND DELIVERY STATUS"** is a **bona fide work** carried out by **[KOWSIKA M] [I A]**, **[B.Sc.INFORMATION TECHNOLOGY]**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[Course Name]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place:

Date:

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	

6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The **Courier Parcel Tracking and Delivery Status** System is a **software application** designed to simplify and enhance the process of managing parcel deliveries. In today's fast-paced world, **efficient logistics and real-time tracking** have become essential for both service providers and customers. This system aims to provide a **reliable, transparent, and user-friendly platform** that allows users to monitor the status of their parcels from dispatch to final delivery.

The primary objective of this system is to **automate the parcel tracking process**, reducing **manual errors** and improving **operational efficiency.** Each parcel is assigned a **unique tracking ID** at the time of registration, which acts as a reference for all future operations. The system maintains detailed information about each parcel, including **sender details, receiver details, current location,** and **delivery status.** Users can easily access this information by entering the tracking ID, enabling **real-time updates** and better communication.

The system typically consists of three main modules: **parcel registration, status update,** and **parcel tracking.** In the parcel registration module, new parcels are added to the system with essential details. The status update module allows administrators or staff to modify the parcel's current state, such as **"Dispatched," "In Transit," "Out for Delivery," or "Delivered."** The tracking module enables customers to view the current status and history of their parcel at any time.

From a technical perspective, the system can be implemented using **programming languages like Java,** with data structures such as **arrays or ArrayList** to store parcel information efficiently. The use of object-oriented programming concepts ensures modularity, scalability, and easy maintenance. Additionally, the system can be enhanced with features like notifications, estimated delivery time, and location tracking.

1. INTRODUCTION

In today's fast-moving world, the need for **efficient and reliable delivery services** has increased rapidly, especially with the growth of online shopping and **e-commerce platforms.** Customers expect not only quick delivery but also accurate and real-time updates about their parcels. Traditional courier systems often fail to meet these expectations due to **manual processes, lack of transparency,** and delays in updating parcel information.

The Courier Parcel Tracking and Delivery Status System is designed to overcome these challenges by providing an **automated** and **user-friendly solution** for managing parcel deliveries. This system helps in **tracking parcels** from the time they are **registered** until they are **delivered** to the receiver. Each parcel is assigned a **unique tracking ID,** which allows users to easily check its **current status** and other relevant details.

The system improves efficiency by **reducing manual work and minimizing errors** in data handling. It allows administrators to update parcel status at different stages such as **registered, dispatched, in transit, and delivered.** At the same time, customers can access real-time information about their parcels, ensuring **better transparency** and communication.

1.1 Objective

The main objective of the **Courier Parcel Tracking and Delivery Status System** is to provide an **efficient** and **automated** way to **track parcels** and **manage delivery information.** The system aims to **reduce manual effort, minimize errors,** and **improve accuracy** in handling parcel data. It ensures that customers can easily access **real-time information** about their shipments using a **unique tracking ID**.
Additionally, it helps courier service providers maintain **proper records,** improve **operational efficiency,** and enhance **customer satisfaction** by offering **reliable** and **timely updates.**

1.2 Overview

The **Courier Parcel Tracking and Delivery Status System** is designed as a **user-friendly application** that stores and manages **parcel-related data** in an organized manner. Each parcel is assigned a **unique tracking ID** at the time of registration, which is used to identify and monitor its journey. The system allows administrators to **add parcel details, update delivery status, and manage records efficiently.** Customers can track **their parcels** at any time by entering the tracking ID, enabling transparency and easy communication.

The system is typically developed using **programming languages such as Java** and uses **data structures** or databases to store parcel information. It follows **object-oriented programming (OOP)** principles, making it **scalable, flexible, and easy to maintain.**

1.3 Features

The **Courier Parcel Tracking and Delivery Status System** includes several important features that make it efficient and user-friendly:

> Parcel Registration: Allows the addition of new parcels with details such as tracking ID, sender, and receiver information.

> Unique Tracking ID: Each parcel is assigned a unique ID for easy identification and tracking.

> Status Update: Enables administrators to update the delivery status (Registered, Dispatched, In Transit, Delivered).

> Real-Time Tracking: Customers can check the current status and progress of their parcels anytime.

> Data Management: Efficient storage and retrieval of parcel information using data structures.

> User-Friendly Interface: Simple and easy-to-use system for both administrators and customers.

>Error Reduction: Minimizes manual errors and ensures accurate data handling.

2. PROGRAM ANALYSIS

Program analysis is the process of examining the **existing courier system** to identify its **problems and requirements for improvement.** In this project, it focuses on understanding the limitations of **manual parcel tracking,** such as **delays, errors,** and lack of **real-time updates.** Based on this analysis, the need for an **automated system** is identified to manage **parcel information** efficiently. It helps in designing a solution that improves accuracy, **reduces manual work,** and provides easy **tracking of parcels** using a **unique tracking ID,** ensuring better performance and **customer satisfaction.**

2.1 Problem Definition

In traditional courier systems, **parcel tracking** is often done manually or using **inefficient methods.** This creates difficulties for both **customers** and service **providers.** Customers are unable to get **real-time updates** about their parcels, leading to **confusion** and **dissatisfaction**. Courier companies also face challenges in **maintaining records** and handling **large volumes of parcels.**

Manual processes increase the chances of **errors** such as incorrect data entry, delays in updating **parcel status,** and loss of important **information**. There is also a **lack of transparency,** making it difficult for customers to track the **exact status** of their shipments. These issues highlight the need for a more **efficient** and **automated system.**

2.2 Project Overview

The **Courier Parcel Tracking and Delivery Status System** is designed to overcome the limitations of **traditional tracking methods.** It is an **automated** system that manages **parcel information** from **registration to delivery.**

Each parcel is assigned a **unique tracking ID,** which is used to **store and retrieve data.** The system allows administrators to **add new parcels, update delivery status,** and **maintain records efficiently.** Customers can **track their parcels** by entering the tracking ID and viewing the **current status.**

The system is developed using **Java programming language** and follows **object-oriented programming (OOP)** concepts. It ensures **efficient data handling** and provides a simple and **user-friendly interface.**

2.3 Scopes and Limitations

Scope

The system helps in managing **parcel details** such as **tracking ID, sender, receiver,** and **delivery status.** It provides **real-time tracking,** improves **transparency,** and reduces **manual errors.** It is suitable for **small to medium-scale courier services** and can be extended with features like **notifications, estimated delivery time,** and **database integration.**

However, the system has certain **limitations**. It may not be suitable for **large-scale operations** without advanced support. **Real-time GPS tracking** is not included in the basic version, and **status updates depend on manual input. Security features** and **scalability** can also be improved with further enhancements.

3. SYSTEM DESIGN

System design is an important phase in **software development** where the **overall structure of the system** is planned. It defines how different **components (modules)** interact to achieve the **desired functionality.** In this system, the design is **modular**, meaning the application is divided into **smaller parts (modules)** for better **organization, easy maintenance, and scalability.**

3.1 Modules

The **Courier Parcel Tracking and Delivery Status System** is divided into different **modules,** each handling a **specific function.** These modules work together to ensure **smooth operation** of the system.

1. Menu Display Module

The **Menu Display Module** is responsible for displaying the **list of available options** to the user. It acts as the **entry point** of the system where users can choose the required operation.

This module typically shows options such as:

> Add Parcel

> Update Parcel Status

> Track Parcel

> Generate Invoice

> Exit

It continuously displays the menu using a **loop** until the user decides to exit the system. This ensures **smooth navigation** and **user interaction.**

2. Order Selection Module

The **Order Selection Module** handles the user's choice from the menu. Based on the selected option, it directs the system to perform the **corresponding operation.**

For example:

If the user selects **"Add Parcel"**, it collects **parcel details**

If **"Update Status"** is selected, it updates the **parcel status**

If **"Track Parcel"** is chosen, it displays the parcel information

This module acts as a **controller** that connects user input with **system functionality.**

3. Bill Calculation Module

The **Bill Calculation Module** is responsible for calculating the delivery charges for the parcel. The calculation may be based on factors such as **parcel weight, distance, or delivery type (standard / express).**

This module ensures:

> Accurate calculation of delivery cost

> Flexibility to include different pricing rules

> Easy modification of pricing logic

It plays an important role in generating the **total** cost for the service.

4. Invoice Generation Module

The **Invoice Generation Module** generates a **detailed bill** or **receipt** for the customer after the parcel is registered or delivered.

The invoice typically includes:

> Tracking ID

> Sender and Receiver details

> Parcel details

> Delivery status

Total cost

This module helps in maintaining **proper records** and provides **proof of transaction** to the customer.

5. Exit Module

The **Exit Module** is responsible for **terminating the application safely.** When the user selects the exit option, the system stops **execution** and closes the program.

This module ensures:

> Proper shutdown of the system

> No loss of data

> **Smooth termination** of the program

4. METHODOLOGY

Methodology refers to the **step-by-step process** followed to develop the system. It explains how the project is **planned, designed, implemented, tested,** and **documented**. This ensures that the system is developed in a **systematic** and **organized manner**.

• Problem Analysis
• System Design
• Program Development
• Testing and Validation
• Result and Documentation

4.1. Problem Analysis

In this phase, the **existing problems** in the **traditional courier system** are studied. **Manual tracking methods, lack of real-time updates, data mismanagement,** and **customer dissatisfaction** are identified as major issues. The need for an **automated system** is clearly understood to improve **efficiency, accuracy, and transparency.**

4.2. System Design

In the **system design phase,** the **overall structure** of the application is planned. The system is divided into different modules such as **menu display, order selection, bill calculation, invoice generation, and exit module.** The design ensures **smooth interaction** and defines how **data flows** within the system. **Object-oriented concepts** are used to make the system **modular** and **easy to maintain.**

4.3. Program Development

In this phase, the **actual coding** of the system is done using a **programming language (Java). Classes, objects,** and **methods** are created to implement functionalities such as **adding parcels, updating status,** and **tracking parcels.**

Data structures (arrays / ArrayList) are used to store **parcel information** efficiently. The program is developed based on the **system design.**

4.4. Testing and Validation

TEST CASE ID	TEST DESCRIPTION	INPUT	EXCEPTED OUTPUT	ACTUAL OUTPUT	RESULT
TC01	Add Parcel	Enter Parcel details	Parcel Added Successfully	Parcel added successfully	pass
TC02	Track Parcel(Valid ID)	Enter valid tracking ID	Display parcel details and status	Parcel details displayed correctly	pass
TC03	Track Parcel(Invalid Parcel)	Enter invalid tracking ID	Display “Parcel not Found”	Correct message displayed	pass
TC04	Update Parcel Status	Enter tracking ID + status	Status updated successfully	Status updated correctly	pass
TC05	Generate Invoice	Enter Parcel details	Invoice with parcel info and cost generated	Invoice generated correctly	pass
TC06	Menu Selection	Choose option from	Perform selected operations	Correct option performed	pass

		menu			
TC07	Exit System	Select exit option	Program terminates	Program closed successfully	pass

4.5. Result and Documentation

This phase involves analyzing the **output of the system** and **documenting** the project. The results show that the system **successfully tracks parcels** and **updates status accurately.** Documentation includes **system design, code, and user instructions,** helping in **understanding** and **maintenance**.

4.6. Algorithm

Algorithm for the **Courier Parcel Tracking and Delivery Status SystemAlo**

Algorithm for parcel Adding parcel

1. Start the program.
2. Prompt the user to enter **Parcel details.**

    > Tracking ID

    > Sender name

    > Receiver name
3. Read all the input details.
4. Store the parcel details in the system.
5. Display **confirmation message** (“Parcel added successfully”).
6. Stop

Algorithm for Parcel tracking

1. Start
2. Prompt the user to enter the **tracking ID.**
3. Read the tracking ID.

4. Check whether the tracking ID exists in the system/
5. If the tracking ID exists, then

    > Display the **Parcel details.**

    > Display the **Current status.**

6. Else

    > Display **“Parcel not founded”.**

7. Stop

4.7. Flowchart

Start
↓
Display Menu
↓
Use selects an option
↓
Platform Operation
(Add / Update / Track / Generate Invoice)
↓
Return to Menu ?
↓
Yes → Go back to Display Menu
↓
No
↓
Exit

↓

End

5. SOURCE CODE

```java
package com.courier.controller;

import com.courier.service.ParcelService;

public class ParcelController {
    public static void main(String[] args) {

        ParcelService service = new ParcelService();
        Scanner sc = new Scanner(System.in);

        while(true) {
            System.out.println("1. Add Parcel");
            System.out.println("2. Upadte status");
            System.out.println("3. Track Parcel");
            System.out.println("4. Exit");

            int choice = sc.nextInt();

            if(choice == 1) {
                System.out.println("Enter Tracking ID: ");
                String id = sc.next();
                System.out.println("Sender: ");
                String sender = sc.next();
                System.out.println("Receiver: ");
                String receiver = sc.next();

                service.addParcel(id, sender, receiver);

            }
            else if(choice == 2) {
                System.out.println("Enter Tracking Id: ");
                String updated = sc.next();
                System.out.println("Enter New Status: ");
                String status = sc.next();

                service.updateStatus(updated, status);

            }
            else if(choice == 3) {
                System.out.println("Enter Tracking ID: ");
                String trackId = sc.next();

                service.trackParcel(trackId);

            }
            else if(choice == 4) {
                System.exit(0);

            }
            else {
                System.out.println("Invalid Choice!");
            }
        }
    }
}
```

```java
package com.courier.service;

import java.util.ArrayList;

public class ParcelService {
    private ParcelRepository repository = new ParcelRepository();
    public void addParcel(String id,String sender,String receiver) {
        Parcel parcel = new Parcel(id, sender, receiver, "Registered");
        repository.save(parcel);
        System.out.println("Parcel Added Successfully!");
    }
    public void updateStatus(String id, String status) {
        ArrayList<Parcel> list = repository.getAllParcels();

        boolean found = false;

        for(Parcel p : repository.getAllParcels()) {
            if(p.getTrackingId().equals(id)) {
                p.setStatus(status);
                System.out.println("Status Updated!");
                found = true;
                break;
            }
        }
        if(!found) {
            System.out.println("Parcel Not Founded");
        }
    }
    public void trackParcel(String id) {

        ArrayList<Parcel> list = repository.getAllParcels();
        boolean found = false;

        for(Parcel p : list) {
            if(p.getTrackingId().equals(id)) {
                System.out.println("Tracking ID: " + p.getTrackingId());
                System.out.println("Sender: " + p.getSender());
                System.out.println("Receiver: " + p.getReceiver());
                System.out.println("Status: " + p.getStatus());
                found = true;
                break;
            }
        }
        if(!found) {
            System.out.println("Parcel Not Founded");
        }
    }
}
```

```java
package com.courier.repository;

import com.courier.model.Parcel;

public class ParcelRepository {
    private ArrayList<Parcel> parcelList = new ArrayList<>();

    public void save(Parcel parcel) {
        parcelList.add(parcel);
    }
    public Parcel findById(String trackingId) {
        for(Parcel p : parcelList) {
            if(p.getTrackingId().equals(trackingId)) {
                return p;
            }
        }
        return null;
    }
    public ArrayList<Parcel> getAllParcels(){
        return parcelList;
    }
}
```

```java
package com.courier.model;

public class Parcel {
    private String trackingId;
    private String sender;
    private String receiver;
    private String status;

    public Parcel(String trackingId, String sender, String receiver, String status) {
        this.trackingId = trackingId;
        this.sender = sender;
        this.receiver = receiver;
        this.status = status;
    }

    public String getTrackingId() {
        return trackingId;
    }

    public void setTrackingId(String trackingId) {
        this.trackingId = trackingId;
    }

    public String getSender() {
        return sender;
    }

    public void setSender(String sender) {
        this.sender = sender;
    }

    public String getReceiver() {
        return receiver;
    }

    public void setReceiver(String receiver) {
        this.receiver = receiver;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }

}
```

6. INPUT AND OUTPUT SCREENS

```
1. Add Parcel
2. Upadte status
3. Track Parcel
4. Exit
1
Enter Tracking ID:
S0101
Sender:
kowsi
Receiver:
sasmitha
Parcel Added Successfully!
1. Add Parcel
2. Upadte status
3. Track Parcel
4. Exit
2
Enter Tracking Id:
S0101
Enter New Status:
delivered
Status Updated!
```

```
1. Add Parcel
2. Upadte status
3. Track Parcel
4. Exit
3
Enter Tracking ID:
S0101
Tracking ID: S0101
Sender: kowsi
Receiver: sasmitha
Status: delivered
```

7. DISCUSSION & CONCLUSION

7.1 Discussion

The **Courier Parcel Tracking and Delivery Status System** improves **efficiency**, reduces **manual errors,** and provides a **structured system** for managing parcel information.

The system has a **user-friendly design** and **modular approach**, allowing easy **parcel tracking** using a **unique tracking ID,** which ensures **transparency** and better **customer satisfaction.**

However, it has some limitations such as dependency on **manual updates** and lack of **GPS tracking** and advanced **security features,** which can be improved in future.

7.2 Conclusion

The **Courier Parcel Tracking and Delivery Status System** is an **efficient solution** that automates **parcel registration, status updates,** and **tracking**, reducing **manual work**.

It provides **real-time information,** improves **accuracy**, ensures **transparency**, and enhances **customer satisfaction.**

The system is **flexible** and **easy to maintain** due to **modular design** and **object-oriented programming,** and can be enhanced with **database integration, GPS tracking,** and **automated notifications.**

8. REFERENCES

The following resources were used during the development of this project:

Books

Herbert Schildt – Java: The Complete Reference

→ *Used for understanding core Java concepts and object-oriented programming.*

E. Balagurusamy – Programming with Java

→ *Helped in learning basic to advanced Java programming techniques.*

Y. Daniel Liang – Introduction to Java Programming

→ *Provided clear explanations of Java fundamentals and data structures.*

Kathy Sierra & Bert Bates – Head First Java

→ *Used for easy understanding of Java concepts with examples.*

Roger S. Pressman – Software Engineering: A Practitioner's Approach

→ *Referenced for software development life cycle and system design concepts.*

Websites

Oracle Documentation

🔗 Visit Oracle Java Official Site

→ Official source for Java programming, tools, and documentation.

Oracle

GeeksforGeeks

🔗 Visit GeeksforGeeks Java Tutorial

→ Provides detailed tutorials on Java, data structures, and programming concepts.

GeeksforGeeks

W3Schools

🔗 Visit W3Schools Java Tutorial

→ Beginner-friendly platform with simple examples and exercises for learning Java.

W3Schools